Support Atelier

ASI

Chapitre 2 : Spring Boot, Maven et persistance des données avec Spring Data JPA

SOMMAIRE

I. Création du projet 2
II. Première entité 3
III. D'autres entités 5
IV. Annexes 5

Veuillez bien lire le document intitulé « **Atelier-Presentation-AutoLoc.pdf** » avant de commencer cet atelier. Ce document est une présentation détaillée du projet **AutoLoc**.

I. Création du projet

1. Ouvrir IntelliJ IDEA → écran d'accueil → cliquer sur « New Project ».
2. Dans le panneau de gauche, sélectionner « Spring Boot ».
3. Renseigner les champs du générateur :
   - Name : **autoloc-api**
   - Location : dossier local du dépôt Git « AutoLoc »
   - Language : **Java**
   - Type : **Maven**
   - Group : **tn.esprit.autoloc**
   - Artifact : **autoloc-api**
   - Package name : **tn.esprit.autoloc**
   - JDK : **17** (ou version supérieure installée)
   - Java : **17**
   - Packaging : **Jar**
   - Configuration : **Properties**

   puis cliquer sur **Next**.
4. Sélectionner les dépendances suivantes (rechercher chaque nom dans la barre de recherche) :
   - Spring Web
   - Spring Data JPA
   - MySQL Driver
   - Lombok
   - Validation
   - Spring Boot DevTools

   puis cliquer sur **Create**.
5. Aller dans File → Settings (ou Preferences sur Mac) → Build, Execution, Deployment → Compiler → Annotation Processors et Cocher « Enable annotation processing ».
6. Vérifier que le plugin **Lombok** est installé (File → Settings → Plugins → rechercher « Lombok ») ou l'installer et redémarrer IntelliJ si besoin.
7. Ouvrir **src/main/resources/application.properties** et renseigner la configuration suivante :

```
spring.application.name=autoloc-api

# Connexion à la base de données
spring.datasource.url=jdbc:mysql://localhost:3306/autoloc_db?createDatabaseIfNotExist=true
spring.datasource.username=root
spring.datasource.password=votre_mot_de_passe

# Hibernate / JPA
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true

# Logs
logging.level.org.hibernate.SQL=DEBUG
logging.level.tn.esprit.autoloc=INFO
```

en mettant le bon mot de passe pour se connecter à la base de données. Consulter l'annexe pour avoir des informations sur la propriété **spring.jpa.hibernate.ddl-auto**.

II. Première entité

8. Sous **tn.esprit.autoloc**, créer les packages suivants (clic droit sur le package → New → Package), qui seront utilisés dès cet atelier ou lors des suivants :

```
tn.esprit.autoloc
├── domain          (entités JPA, énumérations)
├── repository      (interfaces Spring Data JPA)
├── service         (couche métier             )
├── web.controller(contrôleurs REST              )
└── web.dto         (DTO              )
```

9. Dans le package **domain**, créer les énumérations **StatutVehicule** et **CategorieVehicule**.
10. Créer la classe **Vehicule** sans associations :

```java
@Entity
@Table(name = "vehicule")
@Getter
@Setter
@NoArgsConstructor
@AllArgsConstructor
public class Vehicule {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long idVehicule;

    @Column(nullable = false, unique = true, length = 20)
    private String immatriculation;

    @Column(nullable = false, length = 50)
    private String marque;

    @Column(nullable = false, length = 50)
    private String modele;

    @Enumerated(EnumType.STRING)
    @Column(nullable = false, length = 20)
    private CategorieVehicule categorie;

    @Column(nullable = false, precision = 10, scale = 2)
    private BigDecimal tarifJournalier;

    @Enumerated(EnumType.STRING)
    @Column(nullable = false, length = 20)
    private StatutVehicule statut;
}
```

en important les packages suivants :

```java
import jakarta.persistence.*;
import lombok.*;
import java.math.*;
```

Consulter l'annexe pour plus de détails.

11. Ouvrir **AutolocApiApplication.java** et lancer l'application (clic droit → Run), ou bouton ▶ en haut à droite d'IntelliJ.
12. Au niveau du console d'IntelliJ vérifier les logs et résoudre les problèmes s'il y en a. Dans le cas d'utilisation de **MariaDb** comme base de données, ajouter au niveau des propriétés la ligne suivante :

```
spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.MySQLDialect
```

13. Vérifier que vous avez un log de ce type à la fin

```
t.esprit.autoloc.AutolocApiApplication   : Started AutolocApiApplication in 3.229 seconds
```

14. Se connecter à la base **autoloc_db** avec le client SGBD graphique et vérifier que la table **vehicule** a bien été créée, avec les bonnes colonnes et types.

15. Inclure le dossier du projet à **git** (git add .)
16. Valider les changements et pousser le travail sur le dépôt **GitHub** avec un message significatif incluant le chapitre de l'atelier.

III. D'autres entités

17. En utilisant un assistant **IA** (opencode, claude code, codex, cursor ...), faire le même du travail pour les autres entités sans inclure les associations. Aide : créer un dossier de travail différent du dossier du projet puis copier les fichiers générés.
18. Vérifier le code résultat.
19. Relancer l'application et vérifier que toutes les tables sont bien créées (8 entités supplémentaires).
20. Pousser le travail sur le dépôt **GitHub**.

IV. Annexes

Les stratégies spring.jpa.hibernate.ddl-auto

Valeur	Comportement	Quand l'utiliser
none	Hibernate ne touche pas au schéma.	Production, quand le schéma est géré par un outil de migration (Liquibase, Flyway) ou manuellement.
validate	Hibernate compare les entités au schéma existant et lève une erreur au démarrage si elles ne correspondent pas, sans rien modifier.	Environnements sensibles (pré-production, production) : garde-fou pour détecter un décalage entre le code et la base.
update	Hibernate ajoute les tables/colonnes manquantes pour faire correspondre le schéma aux entités, sans supprimer ni renommer les colonnes existantes (une colonne renommée dans le code sera dupliquée, pas migrée).	Développement au quotidien (choix fait pour AutoLoc) : pratique pour itérer rapidement sur le modèle.
create	Hibernate supprime les tables existantes puis les recrée à chaque démarrage, à partir des entités. Toutes les données sont perdues.	Tests ponctuels où l'on veut repartir d'un schéma propre, jamais avec des données à conserver.
create-drop	Comme create, mais supprime en plus le schéma à l'arrêt propre de l'application (utile avec une base embarquée H2).	Tests automatisés (@DataJpaTest, intégration) : base jetable créée puis détruite à chaque exécution.

Les stratégies @GeneratedValue des identifiants

Dans une entité JPA, @Id désigne l'attribut qui sert de clé primaire. L'annotation @GeneratedValue, placée juste à côté, indique à Hibernate comment cette valeur doit être générée automatiquement plutôt que fournie par le code applicatif. Le paramètre strategy accepte une valeur de l'énumération GenerationType, qui compte quatre stratégies.

```
@Id
@GeneratedValue(strategy = GenerationType.IDENTITY)
private Long idVehicule;
```

Les quatre stratégies de GenerationType

Stratégie	Principe	Avantages / limites
IDENTITY	Délègue la génération à une colonne auto-incrémentée gérée par le SGBD (AUTO_INCREMENT en MySQL). Hibernate exécute l'INSERT puis récupère l'identifiant généré.	Simple et nativement supportée par MySQL. Limite : désactive le batching des INSERT chez Hibernate (chaque insertion doit être exécutée immédiatement pour connaître l'id), ce qui peut coûter en performance sur de gros volumes.
SEQUENCE	S'appuie sur une séquence gérée par le SGBD (objet dédié qui distribue des valeurs uniques), interrogée avant l'INSERT.	Compatible avec le batching, généralement la stratégie la plus performante. Non disponible sur MySQL avant la version 8.0 (les séquences n'existaient pas) ; couramment utilisée avec PostgreSQL, Oracle, H2.
TABLE	Simule une séquence à l'aide d'une table dédiée qui stocke le dernier identifiant attribué pour chaque entité.	Fonctionne sur tous les SGBD, y compris ceux sans séquence native. Coûteuse en performance (verrous, requêtes supplémentaires) : à éviter sauf portabilité stricte exigée.
AUTO	Laisse le fournisseur JPA (Hibernate) choisir la stratégie la plus adaptée au dialecte du SGBD configuré.	Pratique pour démarrer ou pour rester portable entre SGBD, mais le comportement réel dépend de la version d'Hibernate et du dialecte : moins prévisible dans un contexte pédagogique où l'on veut illustrer un choix explicite.

Exemples de déclaration

IDENTITY (choix retenu pour AutoLoc avec MySQL)

```
@Id
@GeneratedValue(strategy = GenerationType.IDENTITY)
private Long idVehicule;
```

SEQUENCE (nécessite de déclarer la séquence)

```
@Id
@GeneratedValue(strategy = GenerationType.SEQUENCE, generator = "vehicule_seq")
@SequenceGenerator(name = "vehicule_seq", sequenceName = "vehicule_seq", allocationSize =
1)
private Long idVehicule;
```

TABLE (portable mais coûteuse)

```
@Id
@GeneratedValue(strategy = GenerationType.TABLE, generator = "vehicule_gen")
@TableGenerator(name = "vehicule_gen", table = "id_generator", pkColumnValue =
"vehicule_id")
private Long idVehicule;
```